SVHN Digit Classification

Table of Contents

1. Introduction

This project classifies house-number digits from the Street View House Numbers (SVHN) dataset. Unlike MNIST, SVHN images are cropped from real photographs of house numbers taken from Google Street View, so they vary in lighting, contrast, and digit polarity (light-on-dark vs. dark-on-light), and sometimes include fragments of neighboring digits at the edges of the crop. Thus, training an accurate model on the SVHN dataset is a significantly more challenging and interesting problem.

Three modeling approaches were compared: a multi-layer perceptron (MLP) on both raw and PCA-reduced pixel features, a voting ensemble of MLPs, and a custom convolutional neural network (CNN) built in PyTorch. Preprocessing choices, specifically grayscale conversion and two different digit-polarity normalization heuristics, were evaluated against each model rather than assumed to transfer from one model to another, since (as Section 3 shows) they did not always agree.

2. Data Analysis

The SVHN dataset consists of images of size 32 x 32 with 3 RGB channels. Each image has a label 1-10, where 10 represents the digit 0. Furthermore, the data is divided into a training set and a testing set, with the former containing 73,257 images and the latter containing 26,032 images.

2.1 Grayscaling and Polarity Normalization

Color was converted to grayscale first, collapsing the 3 RGB channels into a single channel via standard luma weights. This alone removes color as a distinguishing feature while preserving the shape information that actually identifies a digit.

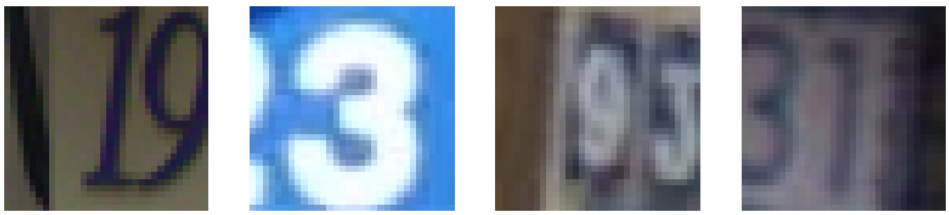

Figure 1: Sample test images before preprocessing.

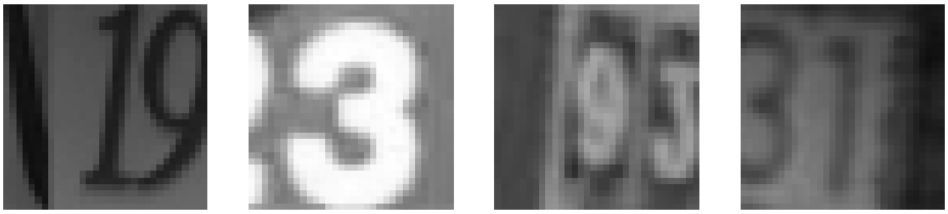

Figure 2: The same images after grayscale conversion.

A second issue was digit polarity: some images show a light digit on a dark background, and others show a dark digit on a light background. Two heuristics were tested to normalize this into a consistent dark-on-light format:

Otsu's method: samples a thin margin around each image's border and computes the threshold that best separates it into two brightness classes which minimizes intra-class variance. If the border is mostly dark, the image is inverted. A low-variance fallback (comparing mean border brightness to the midpoint of the pixel range) is used when the border is too uniform for a meaningful threshold to exist.

Border/interior comparison: compares the mean brightness of a center interior patch (where the digit is expected to be) against the mean brightness of the border margin (background). If the interior is brighter than the border, the image is inverted. Unlike Otsu's method, this uses non-overlapping regions specifically chosen to represent "digit" and "background," rather than analyzing the border in isolation.

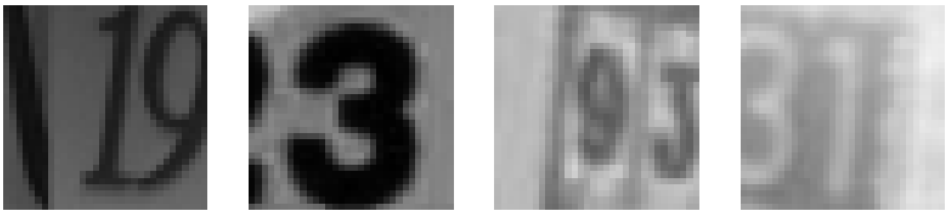

Figure 3: Sample images after Otsu's method normalization.

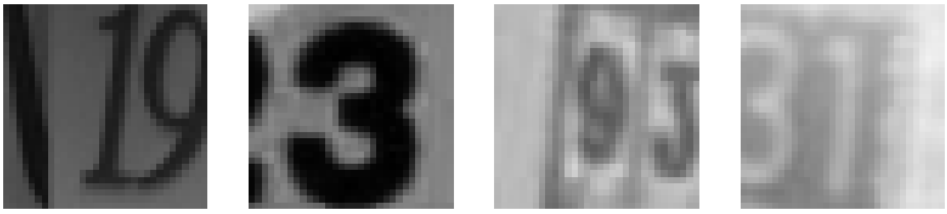

Figure 4: Sample images after border/interior comparison normalization.

2.2 Dimension Reduction

After preprocessing, each 32×32 image is flattened into a 1024-length feature vector and scaled to [0, 1]. Due to the data's high dimensionality, models are more susceptible to overfitting and training is slower. To reduce the dimensionality we apply Principal Component Analysis (PCA) to the unnormalized, Otsu-normalized, and border/interior-normalized feature sets separately, retaining enough components to explain 98% of variance in each case:

Preprocessing	Original features	PCA features (98% variance)
No normalization	1024	86
Otsu normalization	1024	84
Border/interior normalization	1024	84

All three reduce to a similar number of components, consistent with the underlying image structure being the main driver of redundancy in the raw pixel data.

The 98% variance target itself was chosen empirically. A simple MLP (architecture (128, 64)) was trained on PCA-reduced features across four variance targets, 0.90, 0.95, 0.98, and 0.99, to see how test accuracy trades off against the number of retained components.

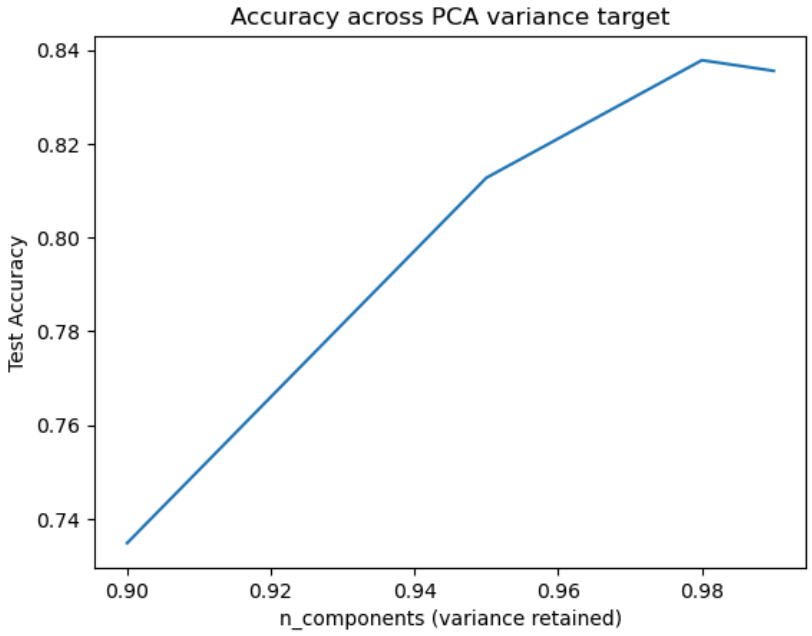

Figure 5: MLP test accuracy across different PCA variance targets.

0.98 was selected as the variance target used throughout this report, as it captures most of the accuracy achievable with more components while still cutting the feature count substantially. The full sweep, including the code, is in Appendix, Section A.3.

3. Model Analysis

3.1 Multilayer Perceptron

A baseline MLP (scikit-learn's MLPClassifier) was trained directly on the flattened pixel features. Hidden-layer architecture was chosen by comparing four options (Appendix, Section A.2). A single hidden layer of width 512 performed best among those tested and was used for the comparison below.

Normalization	Test accuracy
No normalization	0.8079
Otsu method	0.8052
Border/interior comparison	0.7930

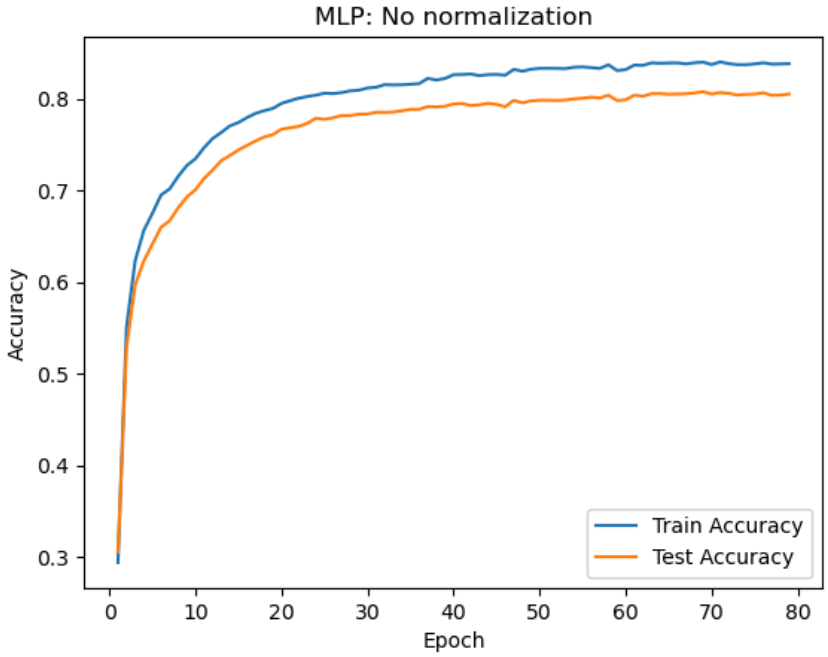

Figure 6: MLP train/test accuracy over epochs, no normalization.

For the MLP, normalization did not help. No normalization outperformed both heuristics, with border/interior comparison performing worst. This is notable because, as shown in Section 3.3, the same three preprocessing options produce the opposite ranking for the CNN. A plausible explanation is that the MLP may already be able to partially learn polarity-invariant patterns from the flattened pixel vectors, while an imperfect normalization heuristic can misclassify images and introduce additional noise.

Additionally, an MLP with two hidden layers of size 128 and 64 was trained on the PCA-reduced features. This architecture was chosen via the sweep in Appendix, Section A.4.

Normalization	Test accuracy (PCA features)
No normalization	0.8355
Otsu method	0.8389
Border/interior comparison	0.8385

PCA improved accuracy across all three preprocessing options relative to the raw-pixel MLP above. Both normalization methods slightly outperform no normalization, though the three results are close enough (within about half a point) that this difference is not strongly conclusive on its own and can be attributed to the non-deterministic nature of training.

3.2 Ensemble Methods

A voting ensemble of three MLPs was tested, combining soft-voted predictions on the PCA-reduced (unnormalized) features. Diversity between members came from varying both architecture and random seed. Specifically, the MLPs had architecture (128, 64), (128,), and (64, 32, 16) and different random_state values. This way, each member starts from different initial conditions and has a different inductive bias.

Model	Test accuracy
Ensemble (soft voting)	0.8534
MLP 1: (128, 64)	0.8351
MLP 2: (128,)	0.8404
MLP 3: (64, 32, 16)	0.8117

The ensemble outperformed every individual member, including the strongest one. This means that architecture and seed diversity produced complementary models here, unlike two alternative ensembling strategies tried and detailed in Appendix, Section A.5:

Feature-splitting ensembles (each MLP sees only a slice of the PCA features): The 2-way split reached 0.8099 test accuracy and the 3-way split reached 0.7931, both below a single MLP trained on the full PCA feature set (0.8355). Splitting features starves each sub-model of information rather than producing complementary views of the digit.

Mixed-algorithm ensemble (MLP + Random Forest + SVM): The ensemble (0.7837) barely beat its own strongest member, the MLP (0.7719), because the Random Forest (0.5719) and

SVM (0.6368) were considerably weaker on their own. Diversity only helps when every member is independently competent; mixing in under-tuned models brings the ensemble down.

3.3 Convolutional Neural Network

A configurable CNN (built directly on torch.nn.Module) was used instead of scikit-learn's MLP, since convolutional layers require 2D spatial input rather than a flattened pixel vector. Each convolutional block applies in order: a 3×3 convolution (stride 1, padding 1, which preserves spatial size so pooling is solely responsible for downsampling), a ReLU activation, 2×2 max pooling (stride 2), and dropout. After all convolutional blocks, the output is flattened and passed through two fully-connected layers: an intermediate layer of width 128 units with ReLU activation and dropout, followed by a final linear layer producing class scores. Training uses early stopping (monitoring test accuracy with a patience window) to prevent overfitting.

The same three normalization options were compared first, using a modest default architecture consisting of two convolutional blocks of 32 and 64 channels, with no dropout:

Normalization	Test accuracy
No normalization	0.8736
Otsu method	0.8676
Border/interior comparison	0.8869

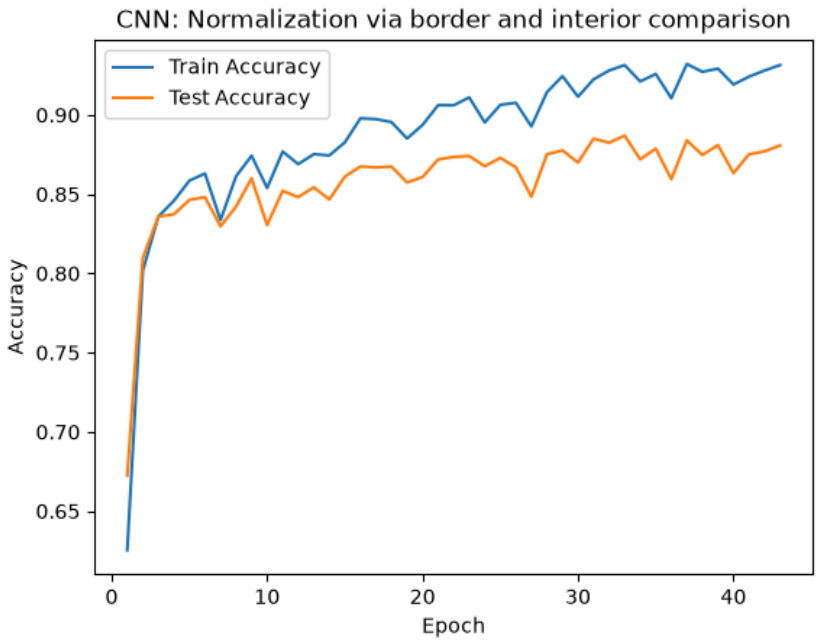

Figure 7: CNN train/test accuracy over epochs, border/interior normalization.

For the CNN, border/interior comparison normalization wins, the opposite ranking from the MLP. Unlike the MLP, a CNN's convolutional filters operate on local spatial patterns; consistent digit polarity likely makes those patterns more consistent across images, which a flattened-vector MLP does not benefit from in the same way.

Various aspects of the CNN architecture were then tuned, including the number of convolutional layers, channel sizes, weight decay rates, and dropout rates (see Appendix, Sections A.6–A.8 for the full sweeps). Structures that increased channel counts with depth generally performed better than other types of structures, and a CNN with channel sizes (32, 64, 128) had the best test accuracy at 0.9038 out of all tested configurations.

Experimenting with dropout rates produced intuitive results: dropout on the first two convolutional layers was neutral-to-harmful, while dropout only on the third convolutional layer gave the largest improvement of any configuration tested, including several multi-layer combinations. This is consistent with early layers extracting features that are more important to preserve, while later layers have more redundant capacity to regularize.

The final CNN configuration combines border/interior normalization, channel_sizes=(32, 64, 128), and dropout applied only after the third convolutional layer (rate 0.5).

4. Final Model Analysis

Based on the comparisons above, the highest test accuracy can be achieved with a CNN trained on border/interior normalized data with the following tuned architecture:

Hyperparameter	Value
Preprocessing	Grayscale + border/interior comparison normalization
Convolutional channels	(32, 64, 128)
Kernel size	3 × 3, stride 1, padding 1
Pooling	2 × 2 max pooling, stride 2, after each convolutional layer
Dropout	0.5 after the third convolutional layer only; no dropout elsewhere
Weight decay (alpha)	1e-3
Optimizer	Adam

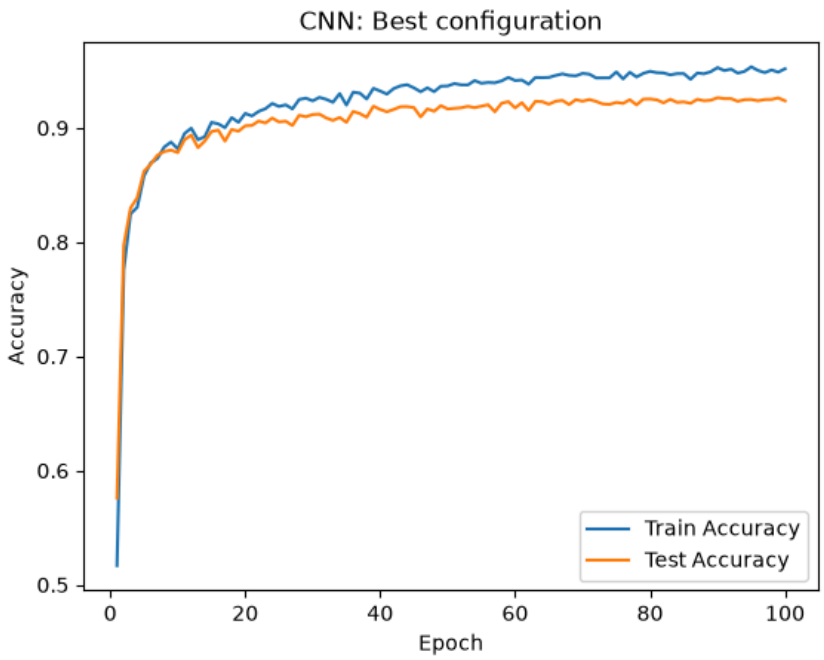

Figure 8: Final CNN train/test accuracy over epochs.

Final test accuracy: 0.9265

Results summary across all approaches evaluated (best result per model/preprocessing combination):

Model	Best test accuracy
MLP, raw pixels (no normalization, 512-unit hidden layer)	0.8079
MLP, PCA features (Otsu normalization, (128, 64))	0.8389
MLP ensemble (architecture + seed diversity)	0.8534
CNN, default architecture (border/interior normalization)	0.8869
CNN, tuned architecture + dropout (final model)	0.9265

The largest single gain in accuracy came from CNN architecture tuning, notably larger than any preprocessing or ensembling choice on its own. This is consistent with the CNN's convolutional structure being much better suited to this image classification task than a flattened-feature MLP, regardless of preprocessing.

Appendix Reference

Full hyperparameter sweeps supporting the choices above, including the MLP and CNN architecture comparisons, the PCA variance-target sweep, the CNN weight decay and dropout sweeps, and the feature-splitting and mixed-algorithm ensembling experiments, are in the accompanying appendix.ipynb notebook. Code and full results for every experiment referenced in this report are in main.ipynb and appendix.ipynb, both included in this repository.